

Guion de exposición C4 — 5 minutos

Fuente: [03_C4_Model_MVP.md](#).

Tiempo ~5–6 min. N3 ~3 min: caso + cajas clave. Flecha discontinua = objetivo.

Nombres al decirlos: bus-workers, retry-worker, BFF del MVP, Adaptador AWS a Azure, Inventario y Reservas — no confundir con el WMS.

N1 — Contexto (~45 s)

(Abrir N1 de una.)

"Leemos la arquitectura en C4 enriquecido con despliegue multinube: primero el contexto de negocio. Este nivel responde quién usa RutaExpress y con quién habla. El Cliente B2B crea órdenes y consulta el seguimiento. El conductor registra entregas y evidencias. Operaciones supervisa el flujo. En el centro está la plataforma, que coordina Azure, AWS y GCP. Por fuera quedan WMS, ERP, Portal y TMS: son sistemas externos y en el MVP los simulamos. Aquí todavía no hablamos de pods: solo de fronteras de negocio."

N2 — Contenedores (~1 min 30 s)

"Bajamos al Nivel 2. Cada nube tiene un rol distinto: no buscamos simetría, sino ubicar cada capacidad donde aporta valor."

Azure — hub operativo

"Azure es el hub operativo. El cliente entra por APIM, nuestro gateway. Un solo AKS compartido ejecuta el BFF del MVP, el OMS, Inventario y bus-workers. Azure SQL guarda el estado y las tablas outbox. Event Hubs es el stream canónico que ya está implementado. Service Bus aporta cola y dead-letter; el recurso existe, pero gran parte del cableado sigue siendo objetivo. Cuando llega un pedido, la Saga reserva inventario por HTTP y el WMS mock se invoca a través de APIM. Quien publica el evento canónico es bus-workers: lee el outbox en SQL y lo manda a Event Hubs, no a Service Bus."

AWS — última milla

"AWS concentra la última milla y las evidencias. Ahí corren mobile-api y retry-worker sobre Fargate. El puente objetivo hacia Azure es este: los eventos salen por SQS, el retry-worker los consume, pasan por EventBridge, el Adaptador AWS a Azure los normaliza y terminan en Event Hubs."

GCP — lectura

"GCP prepara la lectura analítica, al estilo CQRS. El tracking real sobre BigQuery sigue siendo objetivo; hoy el portal de seguimiento del cliente es un mock en APIM."

(Si sobra un segundo:) AKS es el runtime compartido; cada imagen Docker es un deployment distinto.

N3 — componentes críticos (~3 min)

(Texto para leer. Señala al nombrar. Cubrimos el caso con las cajas que importan; el resto solo si preguntan.)

Antes de abrir el primero (~10 s)

"En Nivel 3 hacemos zoom a cuatro contenedores críticos —OMS, Inventario, bus-workers y móvil— porque cada uno resuelve un tramo distinto del flujo. No es la plataforma entera otra vez."

OMS — Caso: el cliente crea un pedido (~45 s)

"Caso: llega un pedido del cliente B2B.

Entra por la Order API. El Correlation Middleware arrastra un ID de traza. El Create Order Handler toma el caso: Dedup Engine e Idempotency Guard evitan duplicados —reintento con la misma clave o pedido repetido distinto—. El Order Aggregate aplica las reglas y la State Machine marca el estado. El Order Repository guarda en Azure SQL y, a la vez, el Outbox Repository deja el evento pendiente: si el bus falla, la intención no se pierde. Después la Saga Orchestrator coordina: con el Inventory Client reserva stock por HTTP y, por Circuit Breaker y APIM, confirma el almacén mock. Si el WMS falla después de reservar, Release libera el stock y el pedido no queda a medias. El evento canónico no se publica aquí: lo completa bus-workers cuando procesa el outbox. La Query API es consulta operativa de soporte, no el tracking masivo de GCP."

Inventario — Caso: reservar o devolver stock (~40 s)

"Caso: la Saga necesita stock ahora; si el almacén falla, hay que devolverlo.

La Reserve API recibe la reserva; el Reserve Handler la ejecuta. Idempotency Guard evita doble reserva por reintento; Optimistic Lock evita que dos pedidos concurrentes pisen el mismo stock. La Reservation Policy y el Inventory Aggregate aplican las reglas de negocio. Si hay que compensar, Release API y Release Handler devuelven el stock. Todo queda en SQL vía Reservation Repository. Hoy el camino vivo es HTTP desde el OMS; el Queue Consumer por Service Bus es el diseño siguiente."

bus-workers — Caso: el evento no se pierde (~45 s)

"Caso: el pedido ya está guardado en SQL; falta avisarle al resto de la plataforma.

En Azure SQL está el outbox: OMS escribió la intención de publicar. El Outbox Poller lee los pendientes —es lo implementado del worker— y el Event Hubs Publisher los manda a Event Hubs, el stream canónico del hub. Si el mensaje falla una y otra vez, cae a dead-letter en Service Bus: no desaparece. En la demo E5 forzamos ese caso y lo reprocesamos con control. Schema Validator, Replay Controller y Backpressure son el diseño para campañas tipo Cyber Days; no los vendemos como ya cableados al 100 %."

Móvil — Caso: entrega sin señal (~45 s)

"Caso: el conductor cierra una entrega sin cobertura y luego sincroniza.

Sin red, la App de Conductores guarda en SQLite cifrado del teléfono. Con red, el tráfico entra por el ALB a la Delivery API; el Delivery Handler coordina el caso. El Evidence Orchestrator arma la firma o foto y el Hash Verifier SHA-256 comprueba integridad; la evidencia queda en S3 cifrada con KMS. DynamoDB guarda el pendiente hacia el hub. El Outbox Relay empuja a SQS —buffer y dead-letter—.

El retry-worker consume SQS, publica en EventBridge y el Adaptador AWS a Azure lleva el evento a Event Hubs. Así la entrega no depende de que Azure esté online en ese segundo."

Cierre (~20 s)

"En conjunto, el MVP demuestra el flujo crítico de órdenes en Azure, con mocks gobernados y el outbox hacia Event Hubs, más la base multinube en AWS y GCP. Lo que pedimos validar son las decisiones y la ruta evolutiva, no una Alternativa A ya cableada por completo."